Distributed Systems & Algorithms Study Guide: 14 Core Data Structures

Oct 25, 2026 · @gg

1. Token Bucket & Leaky Bucket

Both cap how fast requests flow. Token bucket allows bursts up to a limit; leaky bucket forces a smooth output rate.

Goal

Limit request rate to protect a backend and enforce fair use, while deciding how much burstiness to tolerate.

Intuition

- **Token bucket:** a bucket gains tokens at a fixed rate, up to a maximum. Each request spends a token. No token means reject or wait. Idle time banks tokens, so a burst can follow.
- **Leaky bucket:** requests pour into a bucket with a hole in the bottom. Water leaves at a constant rate. If the bucket is full, new water spills (request dropped). The output never exceeds the leak rate.

Detailed design

Parameters: capacity B (max burst) and rate r (tokens per second).

Token bucket, lazy refill (no timer thread):

1. On a request at time t, compute `tokens = min(B, tokens + (t - last_ts) * r)` and set `last_ts = t`.
2. If `tokens >= cost`, subtract `cost` and allow.
3. Otherwise reject. A good `Retry-After` is `(cost - tokens) / r` seconds.

Leaky bucket as a meter: track `level`. On a request, `level = max(0, level - (t - last_ts) * r)`. If `level + 1 <= B`, increment and allow; else reject.

Leaky bucket as a queue: a bounded FIFO takes requests; a worker drains it at rate r. Full queue means drop. Requests wait, so latency grows but the downstream rate stays flat.

Data structure

```
TokenBucket   { tokens: float64, last_ts: int64 }      // ~16 bytes per key
LeakyMeter    { level: float64,  last_ts: int64 }      // same shape
LeakyQueue    { fifo: bounded queue<Request>, drain_rate }
```

Per-user limits use a hash map from key (user, API key, IP) to one bucket. In a cluster, keep each bucket in Redis and run the refill-and-take step in one Lua script so it is atomic. Set a key TTL of about `B / r` so idle buckets expire.

Aspect	Token bucket	Leaky bucket
Burst	Up to B at once	None (meter allows small bursts; queue smooths them)
Output rate	Average r, spiky	Constant r
State	tokens, timestamp	level or queue
Extra latency	None	Yes, in queue form
Best for	APIs that accept bursts	Traffic shaping, steady downstream

Applications

- API gateways and public APIs (AWS API Gateway and Stripe describe token-bucket limiting).
- NGINX `limit_req` is leaky-bucket based, with an optional `burst` queue.
- Linux `tc` Token Bucket Filter for network shaping.
- Guava `RateLimiter` (smooth bursty mode) in JVM services.
- Per-tenant quotas, login throttling, SMS and email send limits.

Problems

- **Atomicity:** read-modify-write races let two nodes spend the same token. Use Lua, atomic counters, or CAS.
- **Clock skew:** lazy refill trusts timestamps. Use one time source (Redis `TIME`) or monotonic clocks.
- **Choosing B and r:** a large B can still overload a downstream that cannot absorb the burst.

- **Memory at scale:** millions of keys need TTLs and eviction; hot keys can bottleneck one Redis shard.
- **Distributed accuracy:** local buckets with periodic sync are fast but can overshoot the global limit.
- **Queue form:** head-of-line blocking and added latency; stale requests may be served after the client gave up.
- **Fairness:** one global bucket lets a noisy tenant starve others. Layer per-tenant buckets under a global one.

2. Bloom Filters

A Bloom filter answers "is X in the set?" in constant time and tiny memory, with no false negatives but a tunable rate of false positives.

Goal

Avoid an expensive lookup (disk, network, database) for items that are almost certainly absent, using far less memory than storing the items.

Intuition

Start with a row of m bits, all 0. To add an item, hash it k ways and set those k bits to 1. To query, hash it the same k ways. If any bit is 0, the item was **definitely never added**. If all are 1, it was **probably added**, because other items may have set those same bits.

Detailed design

1. Pick the expected item count n and target false-positive rate p.
2. Size the filter: m bits and k hash functions (formulas below).
3. `add(x)`: for i in 1..k, set `bits[h_i(x) mod m] = 1`.
4. `contains(x)`: return true only if all k bits are 1.

```
p \approx \left(1 - e^{-kn/m}\right)^{k} \qquad m = -\frac{n \ln p}{(\ln 2)^2} \qquad k = \frac{m}{n}\ln 2
```

Rule of thumb: about 9.6 bits per item gives 1% false positives; about 14.4 bits gives 0.1%. Instead of k independent hashes, derive them from two: `h_i(x) = h1(x) + i * h2(x)` (double hashing).

Data structure

```
BloomFilter { bits: bit array[m], k: int, hash seeds / h1,h2 }
```

No keys or values are stored, so size depends only on n and p, not on item length. Two filters with the same m and k can be merged by bitwise OR (union). Variants fix the weak spots:

Variant	Change	Gain
Counting Bloom	Counters instead of bits	Supports delete
Scalable Bloom	Chain of growing filters	Unknown n
Blocked Bloom	k bits inside one cache line	Faster queries
Cuckoo filter	Stores fingerprints	Delete, often smaller at low p

Applications

- LSM storage engines (Cassandra, HBase, RocksDB, LevelDB, Bigtable) keep a filter per SSTable to skip files that cannot hold a key.
- Browser and security lists: early Chrome Safe Browsing used one to check URLs locally before a server call.
- CDN caches use one to avoid caching objects seen only once.
- Dedup in crawlers and feeds: "have we seen this URL or article?"
- Redis (RedisBloom module) and PostgreSQL (`bloom` index) expose them directly.
- Distributed joins: ship a filter of join keys instead of the keys.

Problems

- **False positives:** never zero. Callers must treat "yes" as "maybe" and verify.
- **No deletion:** clearing a bit may erase other items. Use counting or cuckoo filters.
- **Fixed capacity:** inserting well past n drives p up fast. Size for growth or use scalable filters.
- **No enumeration:** you cannot list members or read items back.
- **Hash quality and adversaries:** weak or predictable hashes inflate collisions; attackers can craft inputs. Use seeded or keyed hashes for untrusted data.
- **Memory access:** k random bit reads mean k cache misses; blocked filters help.
- **Intersection:** bitwise AND only approximates the intersection and raises the false-positive rate.

3. Merkle Trees

A Merkle tree summarizes a large dataset in one root hash, so you can detect any change and prove one item belongs with only about log2(n) hashes.

Goal

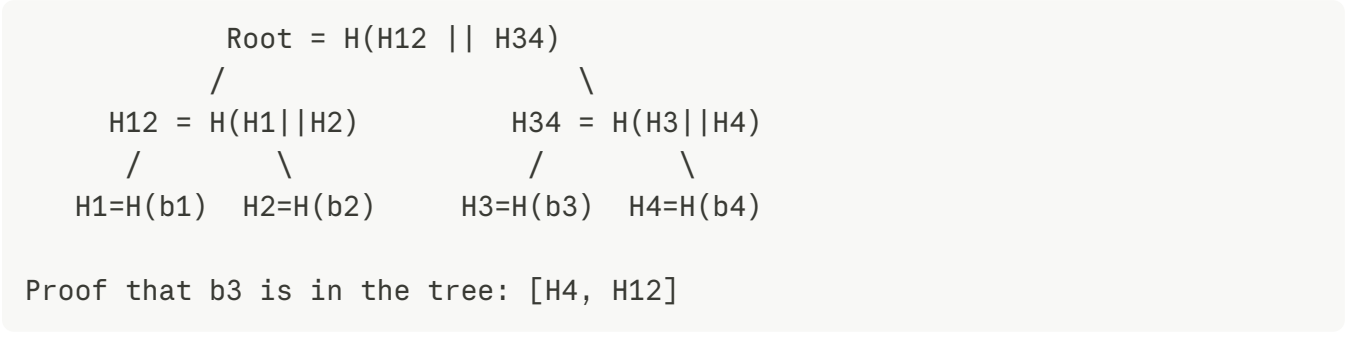
Verify integrity and find differences between two copies of data without sending or comparing everything.

Intuition

Hash every data block. Hash pairs of hashes to make the next level up. Repeat until one hash remains: the root. Change one byte anywhere and every hash on the path to the root changes. Two replicas with equal roots are identical. If roots differ, walk down only the branches whose hashes differ to find the exact bad blocks.

Detailed design

1. Split data into blocks. Leaf = `H(0x00 || block)`.
2. Parent = `H(0x01 || left || right)`. The 0x00/0x01 prefixes (as in RFC 6962) stop a leaf being passed off as an internal node.
3. If a level has an odd count, promote the last node (or duplicate it, as Bitcoin does; see problems).
4. **Membership proof:** give the item plus the sibling hash at each level. The verifier hashes upward and compares with the trusted root. Proof size is O(log n).
5. **Sync / anti-entropy:** compare roots; on mismatch compare children; recurse only into mismatched subtrees. Cost is O(d log n) for d differing blocks.
6. **Update:** change one leaf and rehash its path: O(log n).



Data structure

```
MerkleNode { hash: bytes32, left: *Node, right: *Node, range: [lo, hi] }
```

For a dense tree, store it as an array in heap order (children of i at 2i+1, 2i+2), no pointers needed. For key-value data with a fixed key space, a **sparse Merkle tree** (256 levels, mostly empty subtrees with precomputed default hashes) also proves non-membership. Ethereum uses a **Merkle Patricia trie**, which combines a radix trie with hashing.

Applications

- **Git** stores objects in a Merkle DAG: a commit hash covers its whole tree.
- **Bitcoin** block headers hold a Merkle root of transactions, so light clients verify a payment without the full block.
- **Dynamo-style stores** (Cassandra, Riak, DynamoDB lineage) use them for replica repair.
- **Certificate Transparency** logs prove a certificate is logged and that the log only appends.
- **IPFS, BitTorrent v2, ZFS, Btrfs** use hash trees to verify chunks and detect corruption.
- Software update systems and package registries verify downloads chunk by chunk.

Problems

- **Duplicate-node ambiguity:** duplicating the last node on odd levels let different transaction lists share a root (Bitcoin CVE-2012-2459). Use domain-separated hashing and unambiguous padding.
- **Agreed shape:** both sides must build the same tree over the same ordering and block boundaries, or hashes never match.
- **Cost of repair trees:** rebuilding a tree over large data (Cassandra repair) is heavy on disk and CPU.
- **Inserts shift ranges:** positional trees reshuffle on insert; use key-range trees or prefix tries.
- **Non-membership:** plain Merkle trees cannot prove absence; you need a sorted or sparse variant.
- **Hash dependence:** security rests on a collision-resistant hash; a broken hash breaks every proof.
- **Hot root:** concurrent writers all touch the root path, so updates serialize or must be batched.

4. Raft: Leader Election Deep Dive

Raft keeps a replicated log consistent by electing one leader per term; a candidate needs votes from a majority and an up-to-date log.

Goal

Pick exactly one leader per term, quickly, so a cluster of 2f+1 nodes keeps working through f crashes and never has two leaders committing different data.

Intuition

Time is split into numbered **terms**. Each term has at most one leader. Followers wait for heartbeats. If a follower hears nothing for a random timeout, it assumes the leader is gone, bumps the term, votes for itself, and asks others for votes. Random timeouts make it unlikely that two nodes start at once. A majority vote makes two winners in one term impossible, since any two majorities overlap in at least one node and that node votes once per term.

Detailed design

```
Follower --(election timeout, no heartbeat)--> Candidate
Candidate --(majority votes)-----------------> Leader
Candidate --(sees leader or higher term)-----> Follower
Candidate --(timeout, split vote)------------> Candidate (new term)
Leader    --(sees higher term)----------------> Follower
```

1. **Timeout:** a follower picks a random election timeout (the paper suggests 150-300 ms). Any valid AppendEntries or granted vote resets it.
2. **Become candidate:** `currentTerm++`, `votedFor = self`, reset timer, send `RequestVote(term, candidateId, lastLogIndex, lastLogTerm)` to all peers.
3. **Grant a vote** only if all hold: the request's term is at least the voter's term; the voter has not voted for someone else this term; the candidate's log is **at least as up-to-date** (higher last term wins; if equal, longer log wins).
4. **Win:** a majority of votes. Immediately send empty AppendEntries heartbeats to claim authority.
5. **Lose or tie:** if an AppendEntries from a valid leader arrives, step down. If no one wins, the timeout fires again and a new term starts.
6. **Any message with a higher term** makes the receiver adopt that term and become a follower.

The up-to-date check is what guarantees the new leader already holds every committed entry (Leader Completeness).

Data structure

```
Persistent (fsync before replying): currentTerm, votedFor, log[ {term,
command} ]
Volatile on all nodes:              commitIndex, lastApplied, role,
electionDeadline
Volatile on leader:                 nextIndex[peer], matchIndex[peer]
```

Persisting `currentTerm` and `votedFor` is what stops a restarted node from voting twice in one term.

Applications

- etcd (Kubernetes control plane), Consul, and Nomad use Raft for cluster state.
- CockroachDB, TiKV/TiDB, and YugabyteDB run a Raft group per data range.
- Kafka KRaft mode uses a Raft-based quorum for metadata instead of ZooKeeper.
- RabbitMQ quorum queues replicate with Raft.

Problems

- **Disruptive rejoin:** a partitioned node keeps raising its term, then forces a healthy leader to step down on return. Fix: **Pre-Vote**, a trial round that does not bump terms.
- **Timeout tuning:** too short causes needless elections during GC pauses or slow disks; too long lengthens outages. Keep timeout well above heartbeat interval and network round trip.
- **Stale reads:** a deposed leader may still answer. Use ReadIndex or leader leases (leases assume bounded clock drift) plus check-quorum.
- **Availability needs a majority:** with 2 of 5 nodes alive, the cluster stops accepting writes. That is the safety trade-off.
- **Commit rule subtlety:** a leader only commits entries from older terms indirectly, by committing a current-term entry after them.
- **Membership changes:** adding or removing nodes needs joint consensus or one-at-a-time changes.
- **Leader bottleneck:** all writes pass through one node; shard into multiple Raft groups to scale.

5. Distributed Logs: Redis Streams & Replication Log

A log is an ordered, append-only sequence of records. Position in the log (an offset or ID) is the only coordination readers need, which is why logs sit under messaging, replication, and consensus.

Goal

Give many producers one agreed order of events, let consumers read at their own pace and resume after a crash, and let replicas copy state by replaying the same sequence.

Intuition

Think of a notebook where you only add lines at the end. Each line gets a number. A reader keeps a bookmark (last number read). Replaying from any bookmark rebuilds the same state, so a replica is just a reader that applies every line to its own copy of the data.

Detailed design: Redis Streams

- **Append:** `XADD mystream * field value ...` returns an ID `<ms-timestamp>-<seq>`. IDs are strictly increasing, so ordering is built in.
- **Read:** `XRANGE` for history, `XREAD BLOCK` to tail new entries. The reader remembers the last ID.
- **Consumer groups:** `XGROUP CREATE` makes a group with a `last_delivered_id`. `XREADGROUP` hands each entry to exactly one consumer in the group, so work is shared.
- **Acknowledgement:** delivered but un-acked entries sit in the group's **Pending Entries List (PEL)** with owner and delivery time. `XACK` removes them.
- **Recovery:** `XPENDING` inspects the PEL; `XAUTOCLAIM` or `XCLAIM` moves entries idle too long from a dead consumer to a live one.
- **Retention:** `XADD ... MAXLEN ~ N` or `MINID` trims old entries. The `~` lets Redis trim whole internal nodes, which is cheaper.

Detailed design: Redis replication log

Redis replicates the stream of write commands, tracked by a **replication ID** plus a byte **offset**.

1. A replica connects with `PSYNC <replid> <offset>`.
2. If the master still holds that offset in its **replication backlog** (a fixed-size in-memory circular buffer), it sends only the missing bytes: a partial resync.
3. Otherwise it forks, writes an RDB snapshot, sends it, then streams buffered commands: a full resync.
4. Replication is asynchronous. `WAIT numreplicas timeout` blocks until replicas ack, but this is not a guarantee against loss on failover.

The same idea scales up in Kafka: a partition is a log, followers fetch from the leader, and the **high-water mark** is the offset replicated to all in-sync replicas. Only entries below it are visible to consumers.

Data structure

```
Stream     : radix tree (rax) keyed by entry ID -> listpack nodes holding
many entries
Entry      : { id: ms-seq, fields: [k1,v1,k2,v2,...] }  (field names delta-
encoded in a node)
Group      : { last_delivered_id, PEL: rax(id -> {consumer, delivery_time,
delivery_count}) }
Consumer   : { name, own PEL index, seen_time }
Repl state : { replid, master_repl_offset, backlog: circular byte buffer }
```

Applications

- Task queues and event pipelines on Redis with at-least-once delivery.
- Activity feeds, audit trails, IoT telemetry, lightweight change-data-capture.
- Replication logs in databases (MySQL binlog, PostgreSQL WAL), Kafka topics, and Raft logs all use the same append-and-replay model.
- Event sourcing: current state is a fold over the log.

Problems

- **At-least-once, not exactly-once:** a consumer can crash after processing but before `XACK`, causing redelivery. Make handlers idempotent.
- **Memory-bound retention:** Redis keeps streams in RAM. Trimming bounds memory but permanently drops unread data for slow consumers.
- **Async replication loses writes:** a master can ack a write, crash, and promote a replica that never received it. Use `min-replicas-to-write` and `WAIT` to narrow, not remove, the window.
- **Backlog too small:** a replica that falls behind past the backlog triggers a costly full resync (fork plus RDB transfer).
- **Single-key ordering only:** one stream lives on one cluster shard, so it can become a hot key. Scaling means sharding streams yourself, which gives up global order.
- **PEL growth:** dead consumers leave entries pending forever unless claimed or deleted.
- **Poison messages:** a record that always fails loops forever; track `delivery_count` and move to a dead-letter stream.

6. Lamport Timestamps

A Lamport clock is one integer per process that respects cause and effect: if event A could have influenced event B, then A's timestamp is smaller than B's.

Goal

Order events across machines without synchronized physical clocks, and build a single total order that every node agrees on.

Intuition

Physical clocks drift, so "10:00:01 on server A" and "10:00:00 on server B" prove nothing. What we can trust is causality: a message is received after it is sent. A Lamport clock is a counter that only moves forward, and every message carries the sender's counter. A receiver jumps past it, so an effect always has a larger number than its cause.

Detailed design

Each process keeps a counter `L`, starting at 0.

1. **Local event:** `L = L + 1`.
2. **Send:** `L = L + 1`, attach `L` to the message.
3. **Receive** a message with timestamp `T`: `L = max(L, T) + 1`.

Guarantee (the clock condition): if A happens-before B, then `L(A) < L(B)`.

The reverse does **not** hold. `L(A) < L(B)` does not mean A caused B; they may be concurrent.

Total order: break ties with the process ID. Compare pairs `(L, pid)` lexicographically. This order is arbitrary for concurrent events but identical on every node.

```
P1:  a(1) ---msg(2)--------------->
P2:  b(1)            c = max(1,2)+1 = 3   -> d(4)
P3:  e(1)  f(2)

a -> c holds: 1 < 3.   b and a are concurrent although L(b)=L(a)=1.
```

Data structure

```
LamportClock { counter: uint64 }                // one per process
Message      { payload, ts: uint64, sender: pid }
Order key    = (ts, sender_pid)                 // total order
```

State is a single integer per process, plus the timestamp on each message. Constant space, constant time per event.

Applications

- **Total-order broadcast and replicated state machines:** apply commands in `(ts, pid)` order.
- **Lamport's distributed mutual exclusion:** requests are queued by timestamp; the lowest wins.
- **Operation IDs in CRDTs:** Automerge identifies each operation by a (counter, actor) pair, a Lamport-style stamp.
- **Last-writer-wins registers** and conflict tie-breaking when wall clocks cannot be trusted.
- **Foundation for richer clocks:** vector clocks add concurrency detection; hybrid logical clocks (used in CockroachDB) keep Lamport behavior while staying close to wall time.

Problems

- **Cannot detect concurrency:** two events with different timestamps may be unrelated. Use vector clocks if you must know.
- **Arbitrary tie-breaking:** the order among concurrent events is stable but meaningless, and may disagree with real-world order users saw.
- **No wall-clock meaning:** you cannot ask "what happened in the last 5 minutes".
- **Silent or crashed processes:** a total order needs to hear from every peer (or use a quorum); one quiet node stalls delivery in naive protocols.
- **Counter growth and overflow:** a node with a very large timestamp drags everyone forward. Validate timestamps from untrusted peers.
- **Cost of messages:** Lamport's mutual exclusion needs 3(N-1) messages per critical section, which is heavy at scale.

7. Vector Clocks Deep Dive

A vector clock keeps one counter per process, which captures causality exactly: you can tell "A happened before B", "B before A", or "concurrent".

Goal

Detect conflicting concurrent updates in a replicated system without trusting wall clocks, so conflicts can be kept and resolved instead of silently overwritten.

Intuition

A Lamport clock squeezes all history into one number and loses information. A vector clock keeps the whole summary: "I have seen 3 events from node A, 5 from node B, 0 from node C." If your summary covers mine and has something extra, you are newer. If each of us has seen something the other has not, we are concurrent.

Detailed design

For N processes, each process i keeps a vector `V` of N counters, all starting at 0.

1. **Local event or write:** `V[i] = V[i] + 1`.
2. **Send:** increment `V[i]`, attach a copy of `V`.
3. **Receive** vector `W`: for every j, `V[j] = max(V[j], W[j])`; then `V[i] = V[i] + 1`.

Comparing two vectors A and B:

- A **happened before** B if every `A[k] <= B[k]` and at least one is strictly smaller.
- B happened before A: the mirror case.
- Otherwise they are **concurrent** (a conflict).

```
Start: A=[0,0,0]
Node 1 writes x:           [1,0,0]
Node 2 reads it, writes:   [1,1,0]    (descends from [1,0,0])
Node 3 reads [1,0,0], writes: [1,0,1]  (also descends from [1,0,0])

[1,1,0] vs [1,0,1]: neither covers the other -> concurrent, keep both as
siblings.
A later write that merges both: [1,1,1] + its own increment, e.g. [2,1,1].
```

Data structure

```
VectorClock = map<nodeId, uint64>        // sparse: omit zero entries
VersionedValue { value, clock: VectorClock }
Store: key -> list<VersionedValue>       // several "siblings" when
concurrent
```

On a write, the store drops any stored version whose clock is **dominated** by the new one, and keeps versions that are concurrent with it. On a read, the client may get several siblings and must merge them, then write back with a clock that dominates all of them.

Applications

- **Amazon Dynamo** (the paper) attached vector clocks to object versions to detect concurrent writes, such as divergent shopping carts.

- **Riak** used vector clocks, later **dotted version vectors**, to expose siblings.
- **CRDTs and sync engines** use version vectors to know what each replica has already seen.
- **Collaborative and offline-first apps** use them to find which edits are missing on each device.
- **Debugging distributed traces** to reconstruct a causal order of events.

Problems

- **Size grows with the number of actors:** one entry per writer. If clients are actors, vectors balloon. Dynamo truncated old entries; Riak moved to dotted version vectors.
- **Pruning causes false conflicts or lost causality:** dropping entries can make ordered versions look concurrent.
- **Conflicts need a resolver:** the system only detects them. The application (or a CRDT merge) must decide the result.
- **Sibling explosion:** repeated concurrent writes without merge leave many versions per key.
- **Dynamic membership:** nodes joining or leaving mean retiring or adding vector entries.
- **No real time:** you cannot order concurrent events by "when". Some stores (Cassandra) choose last-write-wins timestamps instead, trading correctness for simplicity.
- **Metadata cost:** every message and stored value carries the vector.

8. Reservoir Sampling

Reservoir sampling picks k items uniformly at random from a stream of unknown length, in one pass, using O(k) memory.

Goal

Get a fair random sample when you cannot store the whole stream or do not know its size in advance.

Intuition

Keep a basket of k items. The first k items go straight in. After that, the i-th item gets a k/i chance of entering, and if it does, it kicks out a random current member. Early items start safe but get bumped more often as the stream grows, and late items enter rarely but with fresh odds. These two effects cancel exactly, so every item ends with the same k/n chance.

Detailed design: Algorithm R

1. Fill `R[0..k-1]` with the first k items.
2. For each later item number i (counting from 1, i > k): pick `j` uniformly from `1..i`.
3. If `j <= k`, set `R[j-1] = item`. Otherwise discard the item.
4. At end of stream, `R` is a uniform sample of size k.

Why it is uniform (induction): after i items, each has probability k/i of being in R. Item i+1 enters with probability k/(i+1). An existing item survives with probability `1 - (k/(i+1)) * (1/k) = i/(i+1)`. So its chance becomes `(k/i) * (i/(i+1)) = k/(i+1)`.

```
k = 1 (pick one at random from a stream)
item1: keep.   item2: replace w.p. 1/2.   item3: replace w.p. 1/3 ...
P(item1 is kept through n=3) = (1/2)(2/3) = 1/3. Items 2 and 3 also end at
1/3.
```

Faster variants:

- **Algorithm L** (Li, 1994) draws how many items to skip before the next replacement, so it makes about O(k(1 + log(n/k))) random calls instead of one per item.
- **Weighted sampling (A-Res, Efraimidis and Spirakis):** give item with weight w the key `u^(1/w)` with u uniform in (0,1); keep the k largest keys using a min-heap.
- **Merging shards:** combine two reservoirs by drawing from each in proportion to how many items each shard saw.

Data structure

```
Reservoir { k: int, seen: uint64, items: array[k] }       // Algorithm R
WeightedReservoir { k, min-heap of (key, item) size k }    // A-Res
```

Memory is k items plus one counter, independent of stream length.

Applications

- **Log and telemetry sampling:** keep a representative set of requests or traces from an unbounded feed.
- **Database statistics:** PostgreSQL `ANALYZE` samples table rows with a Vitter-style reservoir algorithm to build planner statistics.
- **Big-data engines:** Spark's range partitioner samples keys with reservoir sampling to choose partition boundaries.
- **Monitoring percentiles and histograms** from streams.
- **ML pipelines:** draw mini-batches or validation sets from data too large for memory.

- **Interview classic:** "pick a random line from a huge file" (k = 1).

Problems

- **Needs a good random source:** poor or correlated randomness biases the sample. Seed and test it.
- **Treats all items equally in time:** old items have the same weight as new ones. For recency, use sliding-window sampling or exponentially decayed weights.
- **Duplicates:** frequent items appear in the sample in proportion to frequency; use hashing-based (distinct) sampling if you need unique items.
- **Sample size k is fixed:** you cannot grow it later without losing uniformity.
- **Weighted version is trickier:** inclusion probabilities are not simply proportional to weight when k > 1.
- **Distributed merge needs counts:** merging shard reservoirs correctly requires each shard's item count.
- **Estimates have variance:** statistics from k samples have error of roughly 1/sqrt(k); choose k for the accuracy you need.

9. HyperLogLog & Redis PFCOUNT

HyperLogLog (HLL) estimates how many distinct items it has seen, within about 1% error, using kilobytes instead of gigabytes.

Goal

Count unique elements (visitors, IPs, queries) in huge or distributed streams where an exact set would not fit in memory.

Intuition

Flip a fair coin repeatedly. Getting 10 heads in a row is rare: you would expect to need about 2^10 attempts. So if the longest run you ever saw is 10, you probably tried around 1,000 times. HLL hashes each item to a random-looking bit string and tracks the longest run of leading zeros. A single max is noisy, so HLL splits items across many buckets and averages the estimates.

Detailed design

Choose precision p. This gives `m = 2^p` registers.

1. **Add(x):** compute a 64-bit hash `h(x)`.
2. Use the first p bits as the register index `j`.

3. In the remaining bits, count leading zeros and add 1: call it `rho`.
4. Set `M[j] = max(M[j], rho)`.
5. **Estimate:** the harmonic mean of the registers, scaled by a constant.
6. **Merge** two HLLs with the same p: take the register-wise max. The result equals the HLL of the union.

```
E = \alpha_m \, m^2 \left( \sum_{j=1}^{m} 2^{-M[j]} \right)^{-1} \qquad \alpha_m \approx \frac{0.7213}{1 + 1.079/m} \ (m \ge 128) \qquad \text{std. error} \approx \frac{1.04}{\sqrt{m}}
```

Small counts are biased, so implementations switch to **linear counting** (based on the number of empty registers) when many registers are still zero. HyperLogLog++ (Google) adds a 64-bit hash, empirical bias correction, and a sparse form for small sets.

Data structure: Redis

Redis stores an HLL as an ordinary string value, so `GET`, `SET` and persistence all work on it.

- 16,384 registers (p = 14), 6 bits each, so at most **12 KB** per key regardless of count.
- Standard error about **0.81%**.
- Starts in a **sparse** encoding (run-length style) while few registers are set, then converts to **dense** past `hll-sparse-max-bytes` (default 3000 bytes).
- A header caches the last computed cardinality, so repeated `PFCOUNT` on an unchanged key is cheap.

```
PFADD visitors:2026-10-25 u1 u2 u3     # add elements
PFCOUNT visitors:2026-10-25            # estimate distinct count
PFCOUNT visitors:d1 visitors:d2        # union count over several keys
PFMERGE visitors:week d1 d2 d3 ...     # store the union in a new key
```

Applications

- Unique visitors, active users, and unique search queries per day, week, or campaign.
- Network monitoring: distinct source IPs or flows (spotting scans or DDoS).
- Analytics engines: BigQuery `HLL_COUNT`, Presto/Trino `approx_distinct`, Druid, Redshift, and the PostgreSQL `hll` extension.
- Rolling windows by merging per-hour HLLs into per-day or per-week counts without recounting.

Problems

- **Approximate only:** the result can be off by about 1% (and by more for a single query in the tail). Do not use for billing or exact reconciliation.
- **No membership or listing:** you cannot ask whether X was counted or retrieve items.
- **No deletion:** registers only grow. Expiring old items needs per-window HLLs.
- **Intersections are weak:** computing them by inclusion-exclusion (`|A| + |B| - |A union B|`) amplifies error, especially for small overlaps.
- **Hash quality matters:** a poor hash skews the estimate. Redis uses MurmurHash64A.
- **Fixed precision:** merging requires the same p; changing p later means rebuilding.
- **Wrong data type:** Redis returns an error if you run PFADD on a key that is not an HLL string.
- **Adversarial input:** crafted values can bias the sketch if the hash is unkeyed and attackers know it.

10. Byzantine Fault Tolerance: PBFT Deep Dive

PBFT (Castro and Liskov, 1999) lets a fixed group of n = 3f + 1 replicas agree on an order of requests even if f of them lie, collude, or act arbitrarily.

Goal

Keep a replicated service correct and live when some replicas are **Byzantine** (not just crashed): they may send conflicting messages to different peers, forge replies, or stay silent.

Intuition

Crash-fault protocols like Raft trust what a node says. PBFT cannot. So every step needs enough matching votes that, even if f voters are liars, at least one honest node vouches for it. Why 3f + 1: a replica can only wait for n - f replies, since f may never answer. Of those, up to f may be liars, so the honest ones must still outnumber the liars: `n - 2f > f`, i.e. `n >= 3f + 1`. Any two quorums of size 2f + 1 overlap in at least f + 1 replicas, so at least one honest replica sits in both.

Detailed design: normal case (one primary per view)

Phase	Who sends	What it means
Request	Client to primary	Operation plus timestamp, signed

Phase	Who sends	What it means
Pre-prepare	Primary to all	Proposes `(view v, sequence n, digest d)`
Prepare	Each backup to all	"I accept this proposal for (v, n)". Replica is **prepared** after pre-prepare plus 2f matching prepares
Commit	Each replica to all	"I am prepared; commit it". **Committed-local** after 2f + 1 matching commits
Reply	Each replica to client	Result. Client accepts after **f + 1 matching** replies

Prepare ensures all honest replicas agree on the order within a view. Commit ensures that order survives a view change.

Checkpoints: every K requests replicas exchange state digests. When 2f + 1 match, the checkpoint is **stable** and older log entries are discarded.

View change: if a backup's timer expires waiting for a request to execute, it stops accepting messages and sends `VIEW-CHANGE(v+1, last stable checkpoint, prepared set)`. The next primary (`v+1 mod n`) collects 2f + 1 of these and broadcasts `NEW-VIEW` with re-proposed requests, so nothing committed is lost.

Data structure

```
Replica state:
  view v, sequence counter, last stable checkpoint
  log[(v, n)] = { request, digest, pre_prepare, prepares: set<replica>,
commits: set<replica> }
  checkpoints: map<seq, map<replica, state_digest>>
  view_change_msgs: map<new_view, set<VIEW-CHANGE>>
  client table: last reply per client (dedupe retries)
```

The original design used MACs for common-case messages (cheap) and signatures in view change (needed for proofs).

Applications

- **CometBFT / Tendermint** (Cosmos chains) uses PBFT-style voting rounds.
- **HotStuff family** (Diem/LibraBFT, Aptos) keeps PBFT's guarantees with linear message cost per phase.
- **Hyperledger Fabric** offers BFT ordering options (BFT-SMaRt, SmartBFT) next to its default Raft.

- **Permissioned consortium ledgers** and **safety-critical systems** where nodes may be compromised.

Problems

- **O(n^2) messages per request:** prepare and commit are all-to-all, so practical clusters are tens of nodes, not thousands.
- **Costly, complex view change:** the hardest part to implement and to make fast. HotStuff exists largely to simplify it.
- **Primary bottleneck and censorship:** a faulty primary can delay or reorder requests until timers expire.
- **Known, fixed membership:** open networks face Sybil attacks; they need proof of work or stake instead.
- **Timing assumption:** safety holds always, but liveness needs partial synchrony, so timeouts must be tuned.
- **Deterministic execution required:** replicas must compute identical results from the same inputs.
- **Correlated faults:** if all replicas run the same buggy code, f independent faults is a false assumption. Diverse implementations help.
- **More replicas for less tolerance:** 3f + 1 nodes versus 2f + 1 for crash-only protocols.

11. Skip Lists & Redis Sorted Sets

A skip list is a sorted linked list with random "express lanes" above it, giving expected O(log n) search, insert, and delete without any rebalancing. Redis builds its sorted set on one.

Goal

Keep items ordered by a key with fast lookup, insertion, deletion, and range scans, using a simpler algorithm than a balanced tree.

Intuition

On a sorted linked list, finding an item means walking from the start. Add a second list that links every other node, then a third that links every fourth, and so on. Now you ride the top lane until you would overshoot, drop down a lane, and repeat. Instead of building the lanes rigidly, give each new node a random height by coin flips. On average the structure looks like the perfect version.

Detailed design

1. **Level choice:** a new node starts at level 1 and gains each extra level with probability p (1/2 in Pugh's paper, 1/4 in Redis), up to a fixed maximum.
2. **Search(x):** start at the header's top level. While the next node on this level has key < x, move right. When it would overshoot, drop one level. At level 1, the next node is the match or the insert position.
3. **Insert(x):** run the search, remembering the last node visited on each level (the "update" array). Pick a random height, then splice the new node in after those predecessors on each of its levels.
4. **Delete(x):** search, then unlink the node on every level it appears in.
5. **Range scan:** find the start, then walk level 1 for m items: O(log n + m).

Expected time is O(log n); expected space is n / (1 - p) pointers (about 1.33 n at p = 1/4).

```
L3:  H ----------------------> 50 ----------------------> nil
L2:  H --------> 20 ---------> 50 --------> 80 ---------> nil
L1:  H -> 10 -> 20 -> 35 -> 50 -> 65 -> 80 -> 95 -> nil

search(65): L3 to 50, L2 to 80 overshoots -> drop, L1 50 -> 65.
```

Data structure: Redis ZSET

A Redis sorted set pairs two structures over the same members:

- **Hash table:** `member -> score`. Gives O(1) `ZSCORE`.
- **Skip list:** ordered by `(score, member)`. Gives ordered scans and rank queries.

Redis adapts the textbook skip list in three ways:

- Each level stores a **span** (how many bottom-level nodes it skips), so `ZRANK` and rank-based ranges cost O(log n).
- Each node has a **backward pointer** at level 1 so `ZREVRANGE` can walk in reverse.
- Equal scores are ordered by member bytes, so the order is total.

Small sorted sets (up to `zset-max-listpack-entries`, default 128, with short values) use a compact **listpack** instead, and convert to hash table plus skip list when they grow.

```
ZskipNode { member, score: double, backward*, levels[] { forward*, span } }
Zset      { dict: member->score,  skiplist: header, tail, length, level }
```

Command	Cost
`ZADD`, `ZREM`	O(log n)

Command	Cost
`ZSCORE`	O(1)
`ZRANK`	O(log n)
`ZRANGE` / `ZRANGEBYSCORE`	O(log n + m)

Applications

- **Redis:** leaderboards, top-N feeds, priority queues, delayed jobs (score = run-at time), sliding-window rate limits.
- **LevelDB and RocksDB** memtables and **HBase** memstore (`ConcurrentSkipListMap`) keep recent writes in a skip list.
- **Java** **`ConcurrentSkipListMap`** / **`Set`** for lock-free ordered maps.
- **Search engines** use skip pointers inside posting lists to jump over documents.

Skip lists suit concurrency well: an insert touches only nearby pointers, so lock-free variants are easier than for rotating trees.

Problems

- **Probabilistic bounds:** worst case is O(n) with astronomically small probability; no hard guarantee like AVL or red-black trees.
- **Pointer chasing and cache misses:** nodes are scattered in memory, so B-trees often win for on-disk or large in-memory indexes.
- **Memory overhead:** per-node level arrays, plus the extra hash table in Redis, make large ZSETs memory-hungry.
- **Score precision:** Redis scores are doubles; integers above 2^53 lose exactness.
- **Large ranges are expensive:** `ZRANGE` returning many items blocks Redis's single thread; paginate.
- **Hot keys:** one big sorted set lives on one cluster shard.
- **Randomness:** reproducible behavior in tests requires a seeded generator; poor RNG skews heights.

12. AVL Trees vs. Red-Black Trees

Both are self-balancing binary search trees with O(log n) search, insert, and delete. AVL is stricter, so lookups are faster; red-black does less rebalancing work, so updates are cheaper.

Goal

Keep a binary search tree from degrading into a linked list (O(n)) after sorted or adversarial inserts, using small local fixes after each update.

Intuition

A plain BST is fast only if it stays bushy. After each insert or delete, check whether the tree has become lopsided near the changed spot, and if so, **rotate** nodes to even it out. AVL checks "are the two subtrees' heights nearly equal?" at every node. Red-black colors nodes and keeps a color rule that indirectly guarantees no path is more than twice as long as another.

Detailed design

AVL invariant: at every node, `balance = height(left) - height(right)` is -1, 0, or +1.

- After an insert or delete, walk up the path updating heights. At the first node with balance +-2, fix it:
  - Left-Left or Right-Right case: one **single rotation**.
  - Left-Right or Right-Left case: a **double rotation**.
- Insert needs at most one fix (one single or double rotation). Delete may need a fix at every level up the path: up to O(log n) rotations.
- Height is at most about 1.44 log2(n + 2).

Red-black invariants:

1. Every node is red or black; the root is black.
2. A red node has only black children (no two reds in a row).
3. Every path from a node to its leaf nils has the same number of black nodes.

- New nodes are inserted red. If the parent is red, look at the uncle: **red uncle** means recolor and move the problem up; **black uncle** means one or two rotations plus recoloring, then done.
- Insert needs at most 2 rotations; delete at most 3. Recoloring can still travel O(log n) levels, but it is cheap.
- Height is at most 2 log2(n + 1).

```
Right rotation at y (promotes x):

      y                x
     / \              / \
    x   C     =>     A   y
   / \                  / \
  A   B                B   C
```

Data structure

```
AVLNode { key, value, left*, right*, height: int8 }      // or 2-bit balance factor
RBNode  { key, value, left*, right*, parent*, color: 1 bit }
```

Red-black nodes often hide the color bit in an unused pointer bit, so each node costs no extra memory. AVL nodes store a small height or balance field.

	AVL	Red-black
Max height	~1.44 log2 n	~2 log2 n
Lookup	Faster (shallower)	Slightly slower
Insert rotations	At most 2 (1 fix)	At most 2
Delete rotations	Up to O(log n)	At most 3
Extra state per node	Height or balance	1 color bit
Best for	Read-heavy workloads	Mixed or write-heavy workloads

Applications

- **Red-black:** C++ `std::map` and `std::set` (in common implementations), Java `TreeMap` and `TreeSet`, .NET `SortedDictionary`, the Linux kernel `rbtree` (timers, epoll, I/O schedulers), and Java `HashMap` buckets that treeify under heavy collisions.
- **AVL:** read-heavy in-memory ordered indexes and lookup tables where fast search matters more than update speed.
- Both back ordered maps, interval and order-statistic trees (by augmenting nodes with subtree size or max endpoint), and scheduler or timer queues.

Problems

- **Implementation complexity:** red-black delete in particular has many cases; bugs are common. Left-leaning red-black trees tried to simplify it but are not widely used in libraries.
- **Poor cache behavior:** one key per node and pointer chasing; B-trees and skip lists often beat them on modern hardware and on disk.
- **Concurrency:** rotations change structure near the root, making fine-grained locking hard. Skip lists and concurrent B-trees are easier to scale.
- **Rotation cost on heavy-delete workloads** for AVL.
- **Memory:** two child pointers (plus parent) per key is expensive for small keys.
- **Persistence and range-heavy scans** need extra work (path copying, or threading) to behave well.
- **Choosing between them:** differences are constant factors; pick what your library already provides unless profiling says otherwise.

13. BFS & DFS: Graphs and Trees

Breadth-first search explores nearest nodes first and finds fewest-hop paths; depth-first search follows one path to its end before backing up and reveals structure like cycles and ordering. Both run in O(V + E).

Goal

Visit every node reachable from a start exactly once, in a useful order, so you can answer questions about reachability, distance, components, cycles, and dependencies.

Intuition

- **BFS is a ripple in a pond.** It reaches everything 1 step away, then everything 2 steps away, and so on. First arrival at a node is by a shortest path (in an unweighted graph).
- **DFS is exploring a maze with one thread.** Go as deep as possible, and at a dead end back up to the last junction with an unexplored branch.

Detailed design

```
BFS(start):                          DFS(u):
  queue = [start]; seen = {start}      seen.add(u); pre(u)
  dist[start] = 0                      for v in adj[u]:
  while queue:                           if v not in seen: parent[v] = u;
DFS(v)
    u = queue.popleft()                 post(u)
    for v in adj[u]:                   # iterative form: explicit stack
      if v not in seen:
        seen.add(v); dist[v] = dist[u]+1
        parent[v] = u; queue.append(v)
```

Mark nodes as seen **when you enqueue** them in BFS, not when you dequeue, or the same node can be queued many times.

What each gives you:

	BFS	DFS
Frontier	Queue (FIFO)	Stack (LIFO) or recursion
Time	O(V + E)	O(V + E)
Extra space	O(width of graph)	O(depth)
Shortest path (unweighted)	Yes	No
Order on trees	Level order	Pre, in, post-order
Cycle detection	Possible	Natural (back edge)
Topological order	Via Kahn's	Reverse of finish order

DFS edge types (directed graph): tree, back (to an ancestor, means a cycle), forward, and cross. Track three states per node: unvisited, in progress (on the current path), done. Seeing an in-progress node means a cycle.

Useful variants: multi-source BFS (start with several nodes in the queue), 0-1 BFS (deque for edges of weight 0 or 1), bidirectional BFS (search from both ends), iterative deepening DFS (depth limits with DFS memory and BFS-like completeness).

Data structure

```
Graph      : adjacency list  adj[u] = [v1, v2, ...]      // O(V + E) space;
best for sparse graphs
             adjacency matrix  M[u][v]                    // O(V^2); O(1)
edge test; dense graphs
Per node   : seen/visited (bit or set), parent, dist (BFS), discover/finish
times (DFS), color state
Frontier   : deque (BFS) | stack or call stack (DFS)
```

On trees there are no cycles, so the visited set is unnecessary; BFS uses a queue of nodes per level and DFS recursion yields pre-, in-, or post-order.

Applications

- **BFS:** shortest path in unweighted graphs (word ladder, grid mazes), "degrees of separation" in social networks, web crawling by depth, bipartite check, network broadcast, level-order tree printing, nearest-X searches.
- **DFS:** cycle detection, topological sort, connected and strongly connected components (Tarjan, Kosaraju), solving puzzles via backtracking (Sudoku, N-Queens), file system walks, garbage-collector mark phase, finding bridges and articulation points.
- **Both:** connected components, flood fill, reachability checks, dependency analysis.

Problems

- **BFS memory:** the frontier can be huge on wide or branching graphs (an implicit state space grows exponentially).
- **DFS stack overflow:** recursion on deep graphs (a 1,000,000-node chain) crashes; use an explicit stack.
- **DFS finds a path, not the shortest:** do not use it for minimum hops.
- **Weighted edges:** BFS is wrong; use Dijkstra, or 0-1 BFS for weights 0 and 1.
- **Infinite or implicit graphs:** DFS may never return; use depth limits or iterative deepening. Visited sets can exceed memory; a Bloom filter trades exactness for space.
- **Off-by-one mistakes:** marking seen at dequeue time, or forgetting the three-state check for directed cycle detection (a plain visited set gives false cycles).
- **Not unique:** traversal order depends on adjacency order; do not rely on it unless sorted.
- **Hard to parallelize:** DFS is inherently sequential; BFS parallelizes level by level, but frontier sizes are uneven.

14. Kahn's Algorithm

Kahn's algorithm produces a topological order of a directed acyclic graph by repeatedly removing nodes that have no unmet dependencies, and it reports a cycle if it gets stuck.

Goal

Order tasks so every task comes after everything it depends on, and detect circular dependencies, in O(V + E) time.

Intuition

You can only start a task when all its prerequisites are done. So begin with tasks that have none. Finishing one may unlock others: cross it off the list and lower the "waiting on" count of everything that needed it. Whenever a count hits zero, that task is ready. If tasks remain but none are ready, they are waiting on each other: a cycle.

Detailed design

1. Compute `indegree[v]` (number of incoming edges) for every node.
2. Put every node with `indegree == 0` into a queue.
3. While the queue is not empty: pop `u`, append it to `order`, and for each edge `u -> v` decrement `indegree[v]`; if it reaches 0, push `v`.
4. If `len(order) == V`, `order` is a valid topological order. Otherwise the graph has a cycle, and the unprocessed nodes are those on or downstream of a cycle.

```
Edges: shirt->tie, tie->jacket, belt->jacket, trousers->belt, trousers->shoes
indegree: shirt0 tie1 jacket2 belt1 trousers0 shoes1

queue: [shirt, trousers]
pop shirt     -> tie=0 push            order: shirt
pop trousers  -> belt=0 push, shoes=0 push   order: shirt, trousers
pop tie       -> jacket=1              order: ..., tie
pop belt      -> jacket=0 push         order: ..., belt
pop shoes, pop jacket                  order: shirt, trousers, tie, belt,
shoes, jacket
```

Variants:

- **Smallest-first order:** replace the queue with a min-heap for the lexicographically smallest order: O((V + E) log V).
- **Layered order:** process the queue one wave at a time. Each wave is a set of tasks that can run in parallel; the number of waves is the length of the longest dependency chain (the minimum number of rounds).

- **Uniqueness check:** the topological order is unique only if the queue never holds more than one node.
- **Longest path / critical path:** keep `dist[v] = max(dist[v], dist[u] + w)` while popping.

Data structure

```
adj[u]       : list of nodes that depend on u          // adjacency list
indegree[v]  : int array of unmet dependency counts
queue        : FIFO (or min-heap) of ready nodes
order        : output list
```

Time O(V + E) and space O(V + E). The `indegree` array doubles as the progress tracker.

Applications

- **Build systems:** make, Bazel, Gradle, and Ninja order compile steps and run independent ones in parallel.
- **Package managers and installers:** install dependencies before dependents.
- **Workflow schedulers:** Airflow DAGs and data pipelines (dbt, Spark stage planning).
- **Course scheduling** and project planning, including critical-path analysis.
- **Spreadsheet recalculation:** update cells after the cells they reference.
- **Compilers and linkers:** order module initialization and resolve import dependencies.

Problems

- **Reports a cycle, not the cycle:** leftover nodes include everything downstream of the cycle. To print the cycle itself, run a DFS on the leftover subgraph.
- **Many valid orders:** results depend on queue discipline and adjacency order, so tests and builds can vary. Use a min-heap or sort the adjacency lists for determinism.
- **Duplicate edges and self-loops:** duplicates inflate `indegree` unless counted consistently on both sides; a self-loop is an immediate cycle.
- **Static snapshot:** after adding or removing edges, you must recompute, or use incremental topological ordering algorithms.
- **Requires a DAG:** it does not order graphs with cycles; strongly connected components must be condensed first.
- **Memory for huge graphs:** the full graph and indegree array must fit; very large dependency graphs may need partitioning or streaming.
- **Ordering is not scheduling:** it ignores task cost and resource limits, so it gives a legal order, not an optimal schedule.